

Warsha Frontend Performance Review

Current findings, practical fixes, and a recommended path before backend integration.

Current assessment	Good prototype performance, but needs focused bundle and data-layer work before production.
Biggest win	Replace or isolate Recharts, which currently accounts for the largest browser download.
Backend readiness	UI is ready to begin integration; API, authentication, caching, pagination, and authorization still need to be added.

Current production bundle

The project is lighter after the unused feature areas were removed. The core JavaScript and CSS sizes are reasonable for a dashboard application.

Asset	Current size	Assessment
Main JavaScript	About 276 KB / 84 KB compressed	Acceptable
CSS	About 50 KB / 9 KB compressed	Good
Logo image	About 640 KB	Too large for a sidebar logo
Chart shared bundle	About 356 KB / 103 KB compressed	Largest performance issue
Jobs Board	About 89 KB / 27 KB compressed	Reasonable for drag and drop

High-priority fixes

1. Replace or isolate Recharts

The large shared bundle is mostly Recharts, even though its generated file name is misleading. The dashboard needs a simple weekly bar chart, which can be built with lightweight CSS or SVG instead. Replacing it would remove the largest JavaScript cost. If keeping Recharts, configure Vite to load it only with the dashboard chart.

2. Optimize the logo

The current logo is about 640 KB. Resize it to the displayed dimensions and export WebP, AVIF, or an optimized SVG. A sidebar logo should normally be around 20-80 KB.

3. Mount providers only when needed

Bookings, Jobs, Quotes, Services, Reviews, and Settings providers are all mounted globally. Keep authentication global, but move data providers inside the protected app layout or their specific route. This avoids loading and updating data on login/register screens.

4. Keep form state local

Unfinished forms should remain in page hooks, not global context. The quote builder is the main example: saved quotes can be shared, but an unfinished draft should exist only on the Quotes page.

Data and rendering improvements

Context updates

Context action functions and value objects are recreated on updates. Use `useCallback` for actions and `useMemo` for provider values. If a context grows, separate the data context from the actions context so an update affects fewer consumers.

Large collections

Filtering local arrays works for the current mock data. When the backend holds thousands of bookings, quotes, jobs, or reviews, use server-side search, filtering, sorting, and pagination.

Jobs Board

The board currently filters the jobs array and then filters it again for every stage. Wrap derived data in `useMemo` and group jobs in one loop. For very large boards, use virtualized columns or incremental loading.

Dependencies

The project uses `@hello-pangea/dnd`, while `@dnd-kit` packages appear unused. `date-fns` and `react-hook-form` also appear unused. Confirm with the team, then remove unused packages to reduce maintenance and install size.

Backend integration requirements

The frontend does not directly determine how many users the system can handle. Multiple-user capacity depends on the backend, database, hosting, authentication, and API design.

- Create a shared API layer with modules such as bookings, quotes, settings, jobs, and reviews.
- Use consistent backend models with stable IDs; avoid duplicating customer or vehicle display data in every record unless the API intentionally returns expanded data.
- Add loading, error, retry, cancellation, and caching behavior for requests.
- Implement real authentication, server-side authorization, role permissions, and secure sessions.
- Validate every update on the backend, including dates, amounts, status changes, and permissions.
- Handle concurrent edits with timestamps or versioning, and add polling or WebSockets where live updates are needed.

Recommended implementation order

Order	Work	Why it matters
1	Replace or lazy-load Recharts	Largest immediate reduction in downloaded JavaScript
2	Optimize the logo	Fast image-loading improvement
3	Move providers to protected/page routes	Avoid unnecessary data loading and rerenders
4	Add API layer and server-side pagination	Required for real data and larger workshops
5	Memoize contexts and derived lists	Keeps page updates predictable as data grows
6	Remove unused dependencies	Reduces maintenance and project complexity
7	Add production authentication and permissions	Required for secure multi-user operation

Conclusion

Warsha is in good shape as a frontend prototype and is ready to begin structured backend integration. The first four recommended improvements will provide the best balance of faster loading, cleaner architecture, and safer growth to real data and multiple users.